

**Московский Физико-Технический Институт
Физтех-школа Аэрокосмических технологий
Институт Аэромеханики и Летательной Техники**



ФАКТ

**Архитектура Компьютера
и Операционные Системы.
Часть 2. Основы операционных систем**

**Лекция 13: Введение в сетевое
программирование в UNIX. Часть 3/4: TCP/IP**

**Новиков Андрей Валерьевич
д.ф.-м.н.**

Жуковский

TCP/IP

- ❑ TCP/IP — сетевая модель передачи данных в цифровом виде. Описывает способ передачи данных от источника информации к получателю в виде набора протоколов (стэк протоколов)
- ❑ Предполагается прохождение информации через четыре уровня.
- ❑ Название TCP/IP происходит из двух важнейших протоколов семейства — Transmission Control Protocol (TCP) и Internet Protocol (IP), которые были первыми разработаны и описаны в данном стандарте.



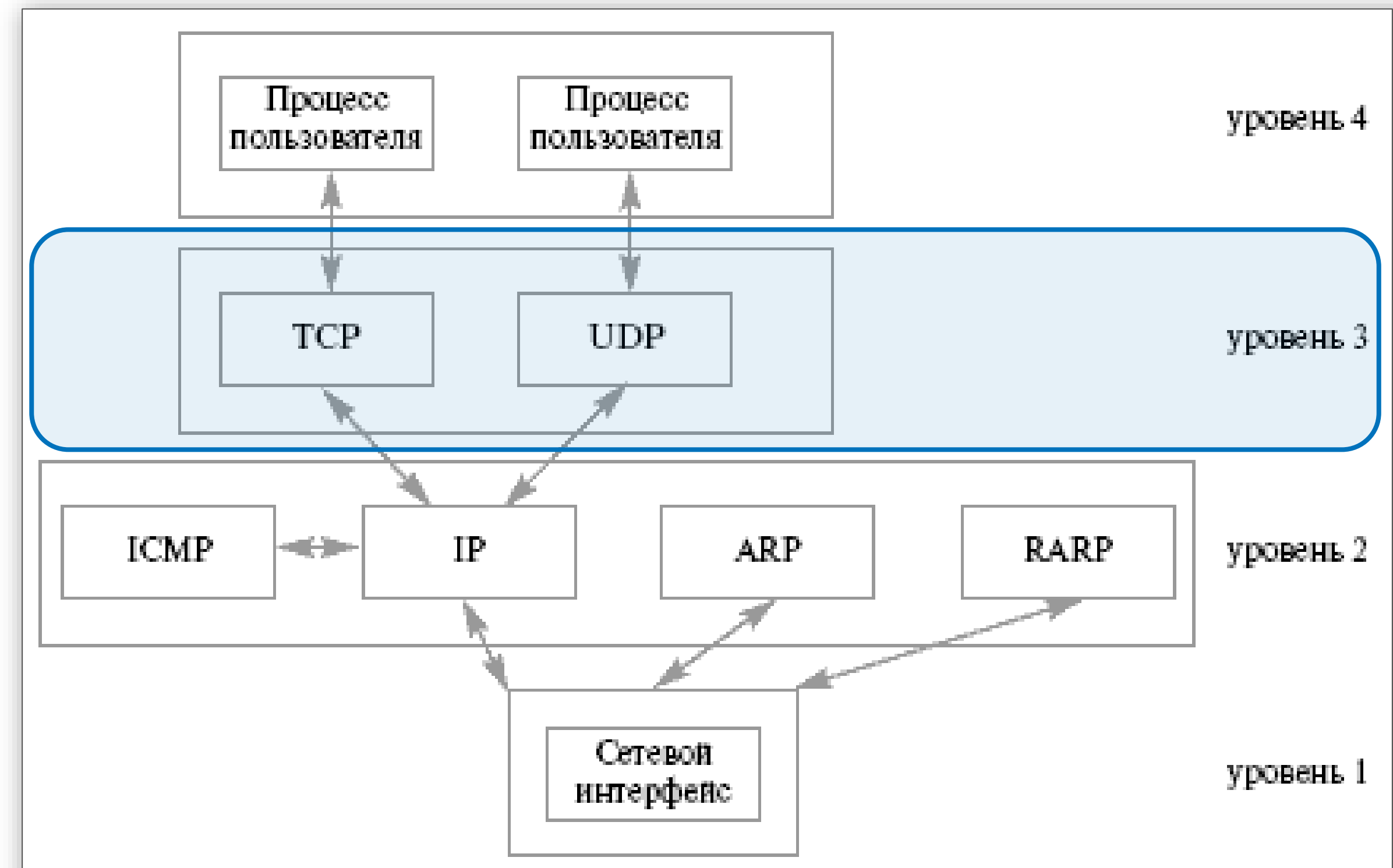
Транспортный уровень TCP/IP

❑ UDP (User Datagram Protocol)

- передача сообщений (*датаграмм*);
- **ненадежный**: не гарантируется доставка датаграмм, сохранение последовательности

❑ TCP (Transmission Control Protocol)

- потоковая модель передачи информации.
- ориентирован на установление логической связи (**connection-oriented**)
- **надежный**: проверка контрольных сумм, передача подтверждения в случае правильного приема сообщения, повторная передача пакета в случае — неполучения подтверждения в течение определенного промежутка времени, правильную последовательность получения информации, полный контроль скорости передачи данных)
- дуплексный способ связи



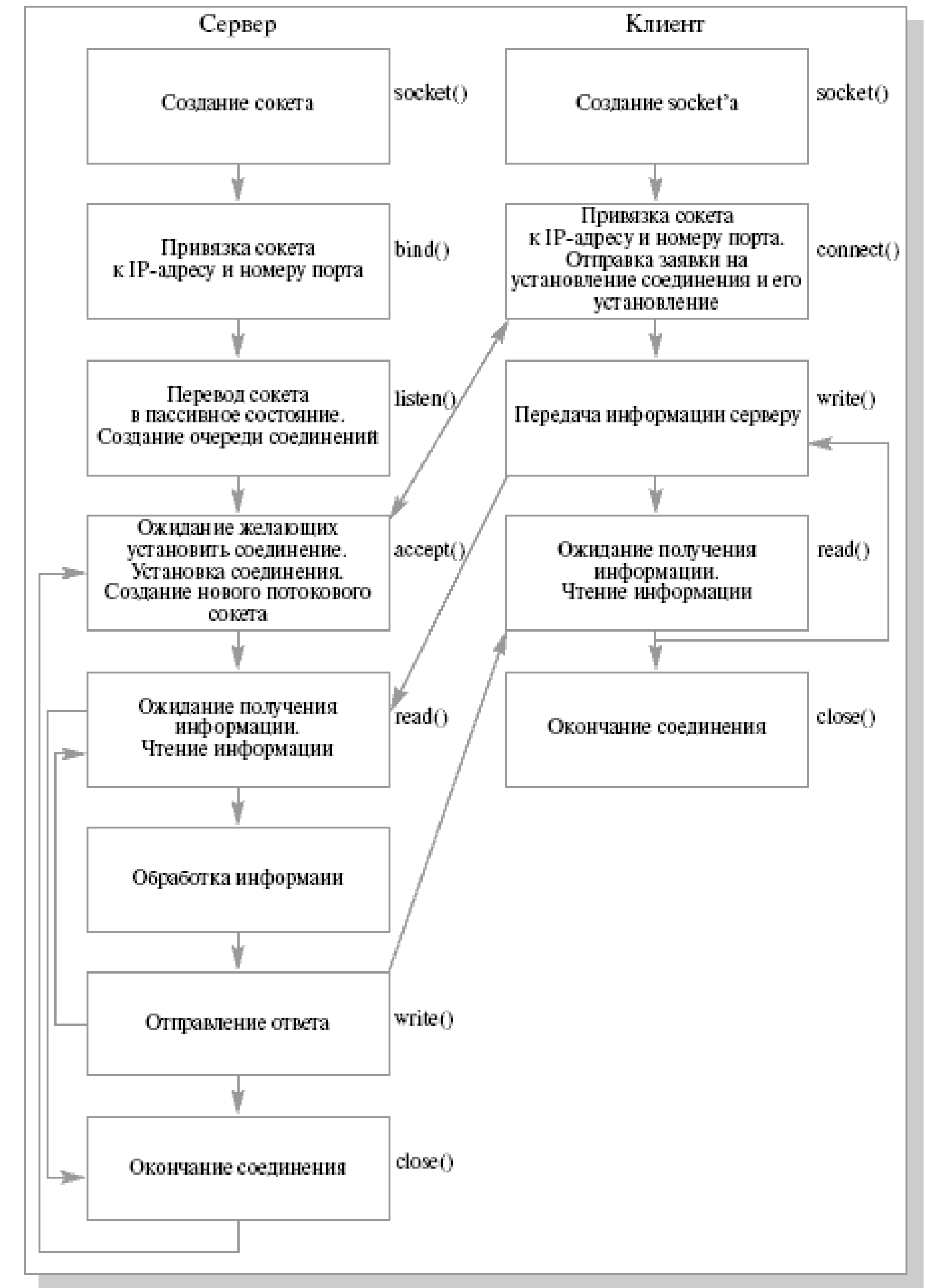
Связь между удаленными процессами с помощью установки логического соединения

ТСР

Взаимодействие клиента и сервера для протокола TCP

Процесс-сервер:

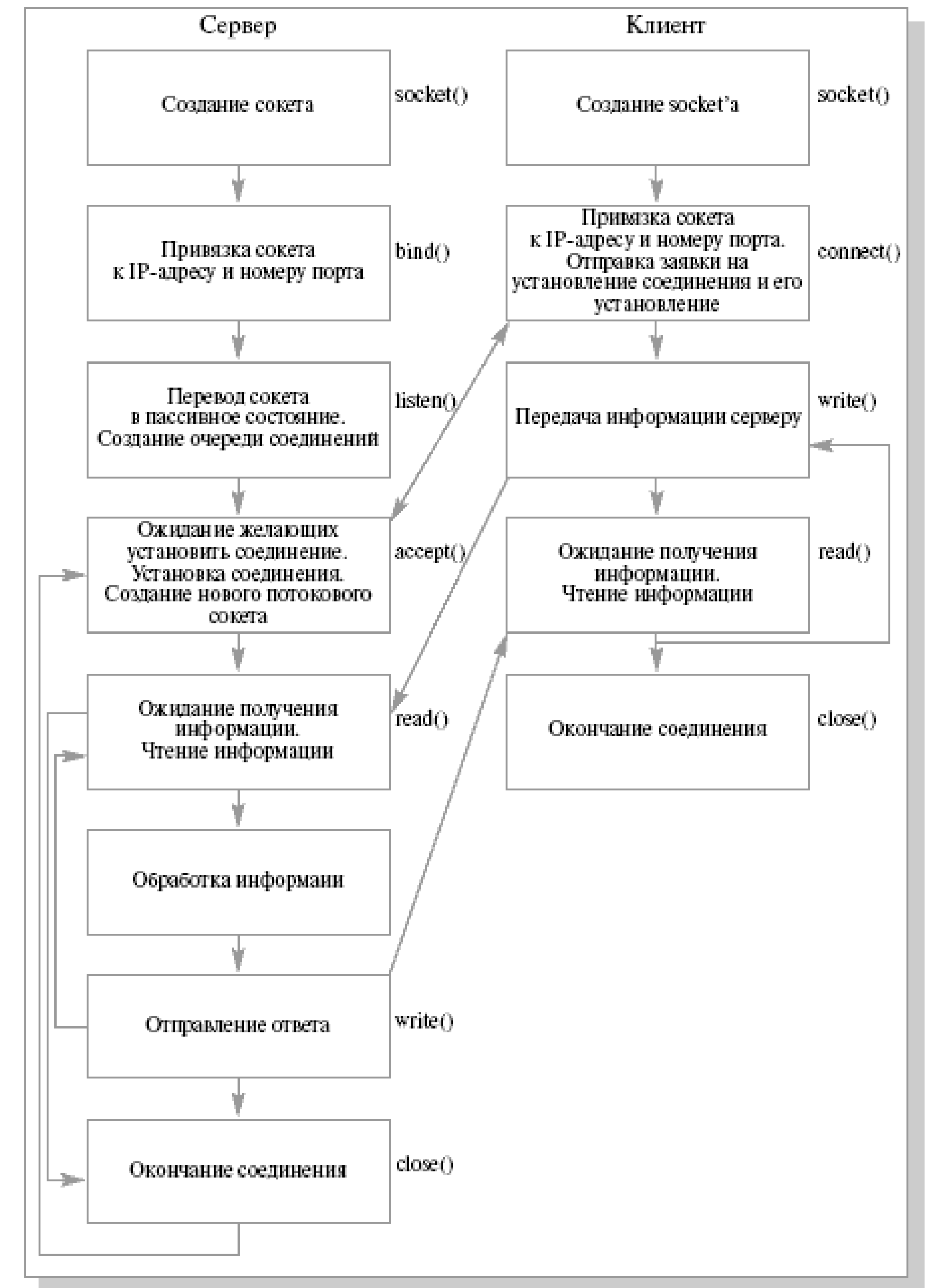
- ❑ `socket()` создать TCP-сокеты (изготовить телефонный аппарат)
- ❑ `bind()` связать сокет с определенным номером порта и IP-адресом сетевого интерфейса (получить номер на АТС) – настроить адрес сокета.
- ❑ `listen()` Разрешить сокету принимать соединения – перевести в пассивный режим.
- ❑ `accept()` принять соединения, узнать адрес клиента изготовить активный потоковый сокет для обмена данными
- ❑ `read/write` – обмениваться данными через активный сокет.



Взаимодействие клиента и сервера для протокола TCP

Процесс-клиент:

- ❑ socket() создать TCP-сокеты («изготовить телефонный аппарат»)
- ❑ connect() = bind() + sendto(): настроить адрес сокета и отправить запрос на соединение.
- ❑ read/write – обмениваться данными через сокет.



Соединение клиента с сервером

```
#include <sys/types.h>
#include <sys/socket.h>

int connect(int sockd,
            struct sockaddr *servaddr,
            int addrlen);
```

Организация связи клиента с сервером. Обычно устанавливают логическое соединения, хотя можно применять и при связи на датаграммах.

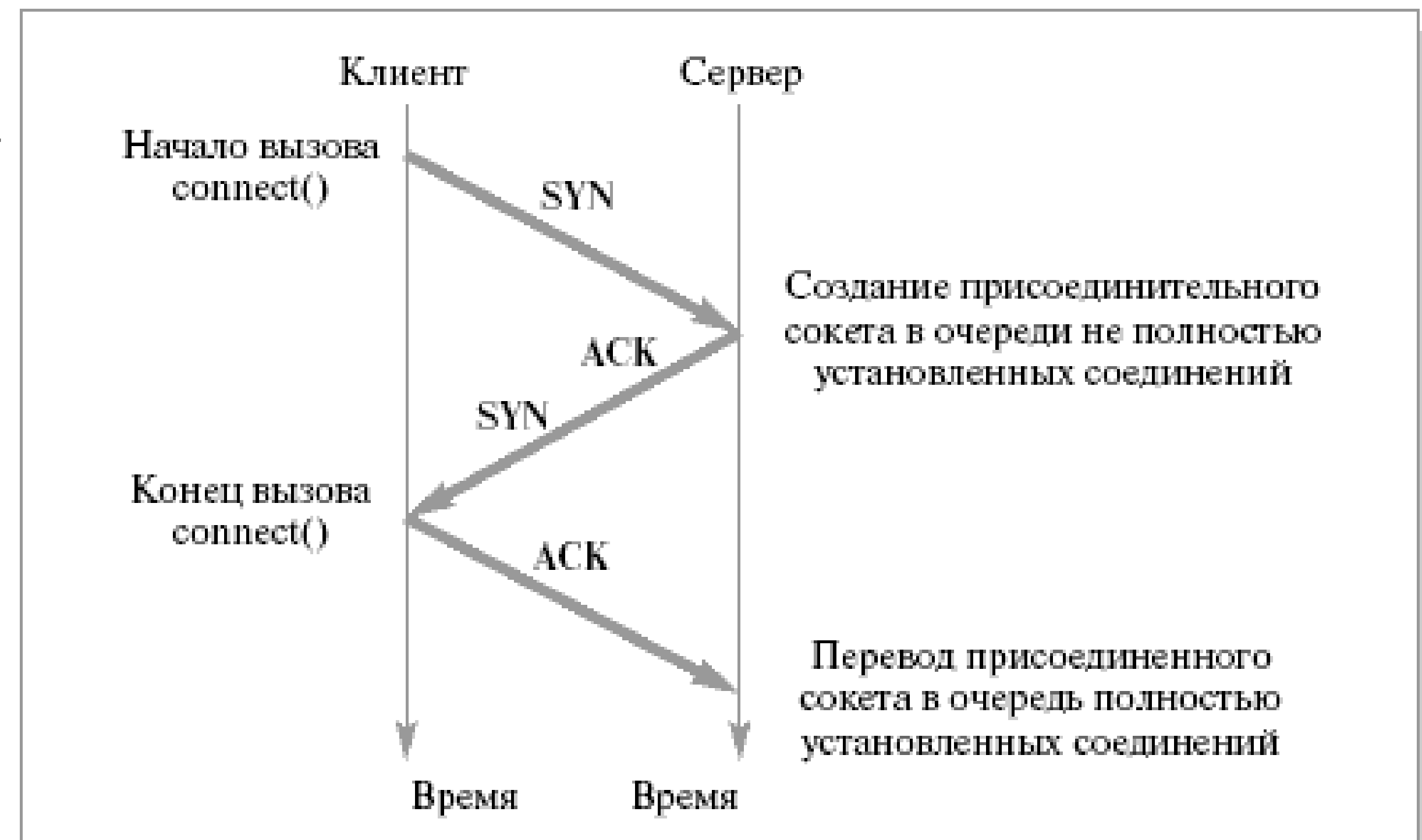
Блокируется до установления соединения или до окончания таймаута.

- ❑ **sockd** — дескриптор сокета
- ❑ **servaddr** — адрес сокета сервера
- ❑ **addrlen** — фактическая длина структуры с адресом
- ❑ Возвращаемое значение:
0 - успех, < 0 - ошибка

Установка виртуального соединения

Трёхэтапное рукопожатие (three-way handshake)

- ❑ 1. Клиент посылает серверу специальный пакет **SYN** (от слова synchronize – синхронизировать), который содержит начальный номер для последующих пакетов данных, который будет использовать клиент.
- ❑ 2. Сервер подтверждает получение пакета **SYN** от клиента. Создаёт сокет с не полностью установленным соединением. Отправляет клиенту свой пакет **SYN** с начальным номером для пакетов данных, в виде единого пакета с сегментами **SYN** и **ACK** (от слова acknowledgement – подтверждение).
- ❑ 3. Клиент пакетом **ACK** подтверждает прием пакета от сервера. Сервер переводит свой сокет в состояние полностью установленного соединения



Ожидание соединения на сервере

```
#include <sys/types.h>
#include <sys/socket.h>

int listen(int sockd,
           int backlog);
```

Перевод TCP-сокета в пассивное (слушающее) состояние и создание очереди для порождаемых при установлении соединения сокетов, находящихся в состоянии не полностью установленного соединения и полностью установленного соединения.

- ❑ **sockd** — дескриптор сокета
- ❑ **backlog** — максимальный размер очередей для сокетов, находящихся в состояниях полностью и/или не полностью установленных соединений (в зависимости от версии Unix).
- ❑ Возвращаемое значение:
0 - успех, < 0 - ошибка

Приём соединения на сервере

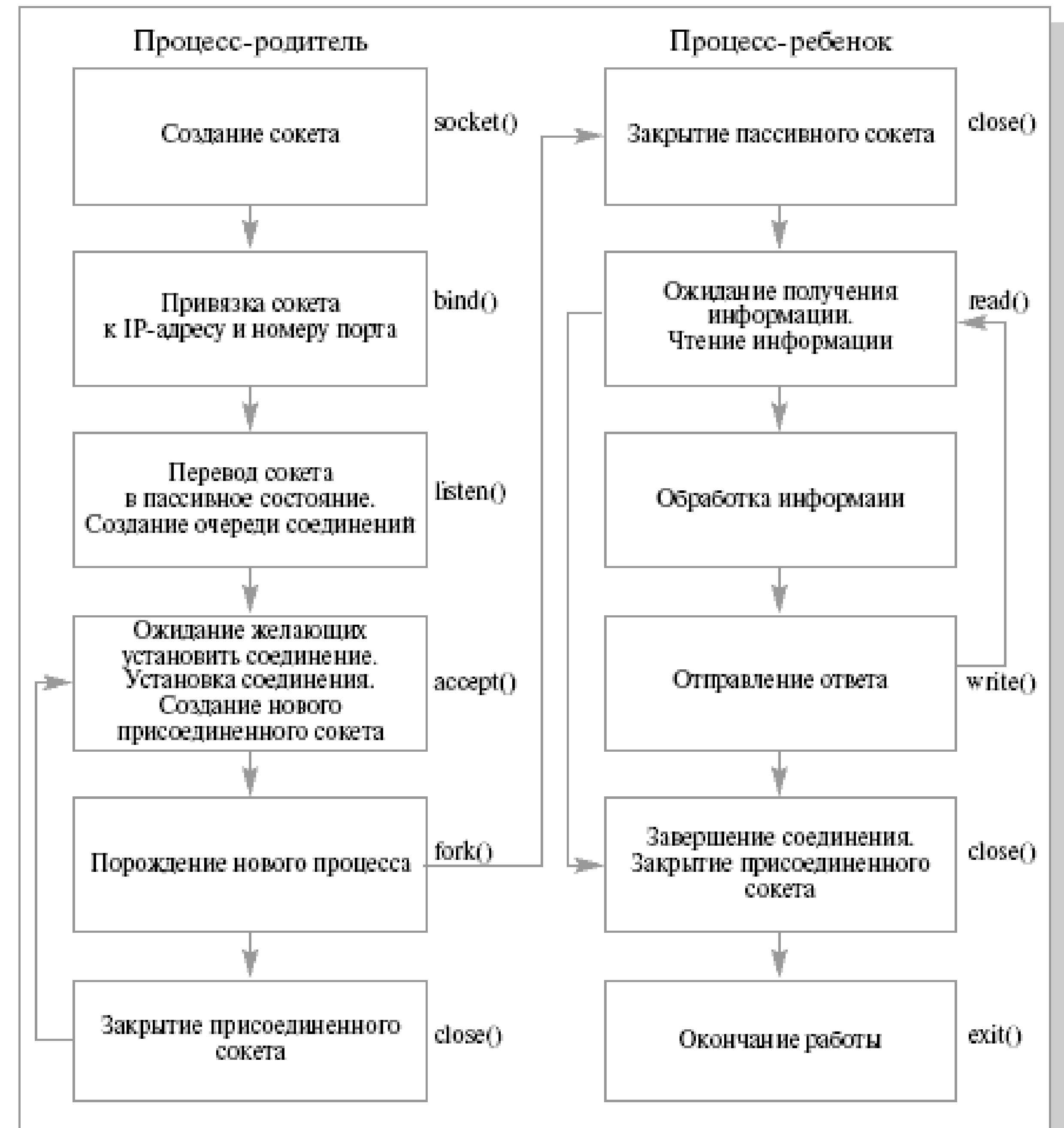
```
#include <sys/types.h>
#include <sys/socket.h>

int accept(int sockd,
           struct sockaddr *cliaddr,
           int *clilen);
```

Приём полностью установленного соединения. Возвращает дескриптор для первого присоединенного сокета из очереди соединений, одновременно удаляя его из очереди. Или ожидает появления соединения.

- ❑ **sockd** — дескриптор сокета
- ❑ **cliaddr** — адрес сокета отправителя, может быть NULL
- ❑ **addrlen** — длина структуры с адресом. До вызова – максимальная длина, после вызова – фактическая длина
- ❑ Возвращаемое значение:
 - ≥ 0 – дескриптор сокета для обмена данными,
 - < 0 - ошибка

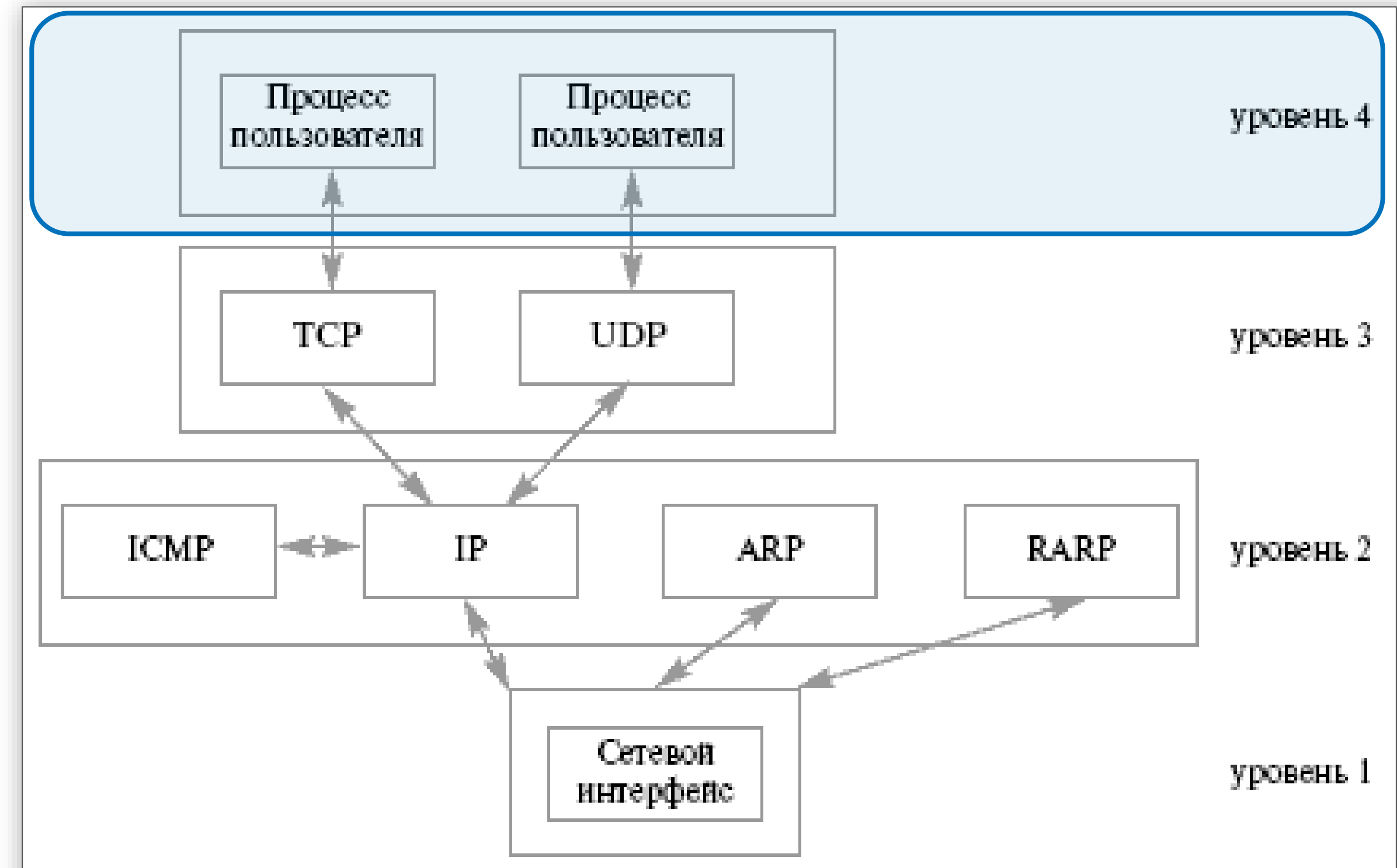
Параллельная обработка клиентов на сервере



Прикладной уровень (уровень приложений/процессов)

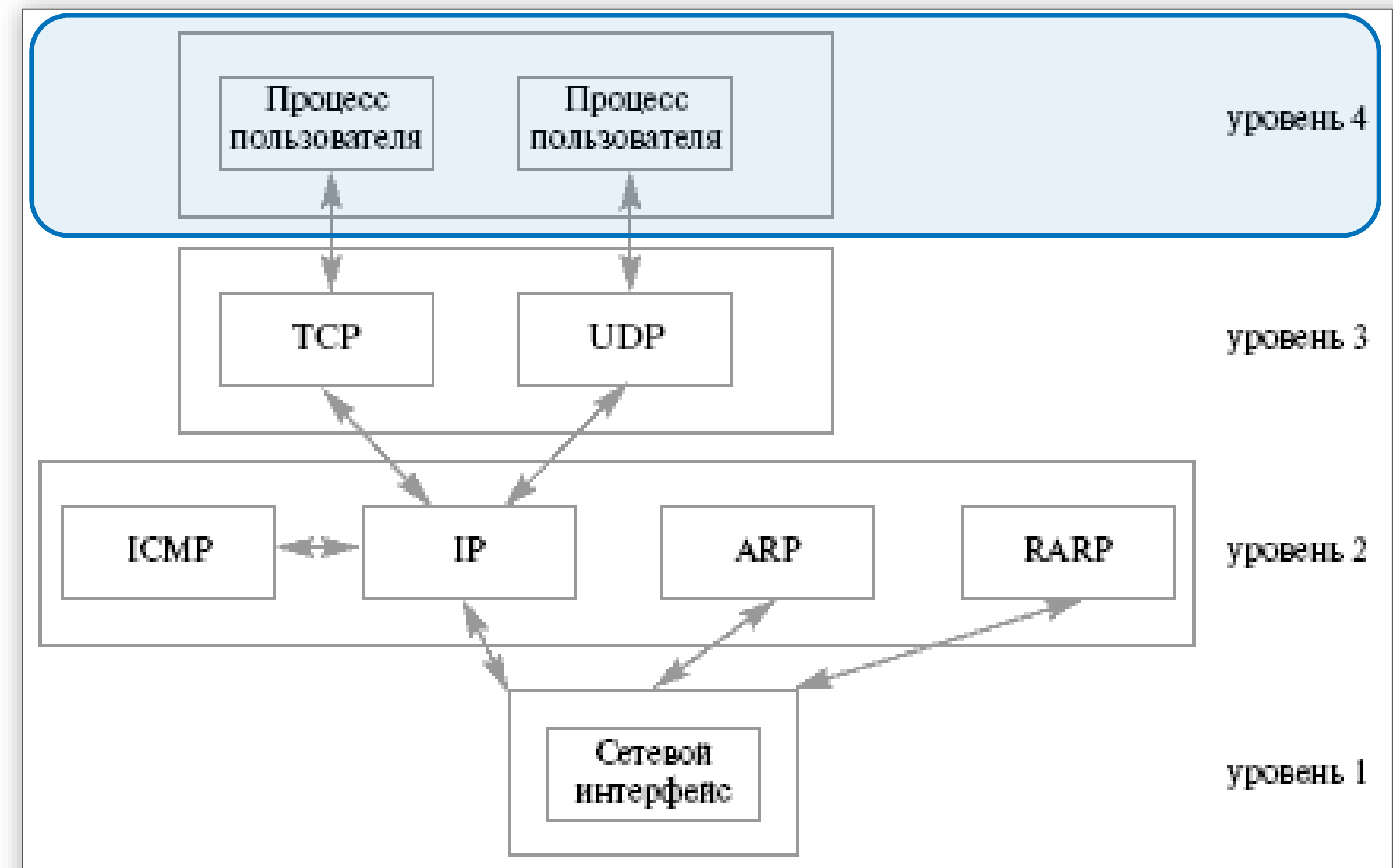
□ У каждого приложения собственный протокол обмена информацией:

- **HTTP** (hyper text transfer protocol), **HTTPS** (HTTP Secure) для интернет браузера;
- **FTP** (file transfer protocol) для передачи файлов;
- **SMTP** (simple mail transfer protocol), **POP** (post office protocol), **IMAP** (Internet Message Access Protocol) для электронной почты;
- **SSH** (безопасное соединение с удалённой машиной)
- **DNS** (Domain Name System) (преобразование символьных имён в IP-адреса)
- многие другие.



Прикладной уровень (уровень приложений/процессов)

- ❑ Протоколы работают поверх TCP или UDP и привязаны к определённому порту:
 - **HTTP** на TCP-порт **80**,
 - **FTP** на TCP-порт **20** (для передачи данных) и **21** (для управляющих команд),
 - **SSH** на TCP-порт 22,
 - **DNS** на порт UDP (реже TCP) 53,
 - ...
- ❑ Порты определены Агентством по выделению имён и уникальных параметров протоколов (IANA - *Internet Assigned Numbers Authority*).



Пример. Протокол HTTP

- ❑ Клиент и сервер обмениваются HTTP-сообщениями.
- ❑ Сообщения текстовые, состоят из трёх частей:
 1. Starting line — тип сообщения;
 2. Headers — характеризуют тело сообщения, параметры передачи и прочие сведения;
 3. Message Body — непосредственно данные сообщения. Обязательно должно отделяться от заголовков пустой строкой.

Клиент (браузер):

```
GET /wiki/Main_Page HTTP/1.1
Host: ru.wikipedia.org
Accept-language: ru,en
<пустая строка>
```

←(1)

←(2)

Ответ сервера:

```
HTTP/1.1 200 Ok
Date: Wed, 07 Dec 2022
...
Content-length: 12345
<пустая строка>
<html><head>...
```

←(1)

←(2)

←(3)₁₄

Литература

1. В.Е. Карпов, К.А. Коньков - Основы операционных систем // М.: Национальный открытый университет «ИНТУИТ», 2016
<https://intuit.ru/studies/courses/2192/31/info>
2. В.Е. Карпов, К.А. Коньков - Основы операционных систем. Практикум // М.: Национальный открытый университет «ИНТУИТ», 2016
<https://intuit.ru/studies/courses/2249/52/info>
3. Google 😊

